

Practical 8

Task 1

Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

```
#include <stdio.h>
#include <stdlib.h>
int global_var = 10; // Global variable
static int static_var = 20; // Static variable
void function()
{
    printf("Code address : %p\n", (void *)function);
}
int main()
{
    int stack_var = 30; // Stack variable
    int *heap_var = malloc(sizeof(int)); // Heap variable
    *heap_var = 40;
    printf("Memory Addresses:\n\n");
    function();
    printf("Global address : %p\n", (void *)&global_var);
    printf("Static address : %p\n", (void *)&static_var);
    printf("Heap address : %p\n", (void *)heap_var);
    printf("Stack address : %p\n", (void *)&stack_var);
    free(heap_var);
    return 0;
}
```

OUTPUT:

```
root@Tejasri:~# nano program.c
root@Tejasri:~# gcc program.c -o program
root@Tejasri:~# ./program
Memory Addresses:

Code address : 0x58fbd8bf21c9
Global address : 0x58fbd8bf5010
Static address : 0x58fbd8bf5014
Heap address : 0x58fbf84b72a0
Stack address : 0x7ffd3609d61c
root@Tejasri:~#
```

Task 2

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    int *data = malloc(5 * sizeof(int));
    if (data == NULL)
    {
        perror("malloc");
        return 1;
    }
    for (int i = 0; i < 5; i++)
        data[i] = i + 1;
    printf("Parent before fork:\n");
    printf("Data address = %p\n", (void *)data);
    printf("Data[0] = %d\n\n", data[0]);
    pid_t pid = fork();
    if (pid < 0)
    {
        perror("fork");
        free(data);
        return 1;
    }
    if (pid == 0)
    {
        /* Child process */
        printf("Child before modification:\n");
        printf("Data address = %p\n", (void *)data);

```

```

        /* Modifying data triggers Copy-on-Write */
        data[0] = 100;
        printf("Child after modification:\n");
        printf("Data address = %p\n", (void *)data);
        printf("Data[0] = %d\n\n", data[0]);
        free(data);
        exit(0);
    }
    else
    {
        /* Parent process */
        wait(NULL);
        printf("Parent after child modification:\n");
        printf("Data address = %p\n", (void *)data);
        printf("Data[0] = %d\n", data[0]);
        free(data);
    }
    return 0;
}

```

OUTPUT:

```
root@Tejasri:~# nano cow.c
root@Tejasri:~# gcc cow.c -o cow
root@Tejasri:~# ./cow
Parent before fork:
Data address = 0x56aaea9be2a0
Data[0] = 1

Child before modification:
Data address = 0x56aaea9be2a0
Data[0] = 1

Child after modification:
Data address = 0x56aaea9be2a0
Data[0] = 100

Parent after child modification:
Data address = 0x56aaea9be2a0
Data[0] = 1
root@Tejasri:~# |
```